

Enfin maîtriser les Expressions Rationnelles

Par [Christophe Porteneuve](#) 

Date de publication : 30 janvier 2014

Une couverture exhaustive des expressions rationnelles a jusqu'ici fait partie de l'atelier **JS Puissant**. Elle occupait bien 1h, voire 1h30, en matinée.

C'est un sujet qui me tient véritablement à cœur, pour les raisons que je vais développer dans les deux premiers titres. Mais on m'a quelquefois fait remonter, très justement, qu'une telle couverture, au sein d'un atelier de 8h, est tout à fait hors de proportion et que le temps libéré pourrait être utilement mis à profit en entrant plus dans le détail d'autres aspects (programmation fonctionnelle, structuration de code...).

Afin de ne pas simplement cesser de transmettre ces connaissances et de « convertir » les gens au bon usage des expressions rationnelles, j'ai donc opté pour l'approche inverse : sortir ce contenu des formations pour en faire un contenu librement accessible sur le site de JS Attitude !

Qui sait ? Peut-être cet article, s'il est suffisamment bien fichu, deviendra-t-il, à son petit niveau, une sorte de référence francophone sur ce sujet, comme le sont désormais des articles de **Git Attitude** sur les **clés SSH** ou **l'installation de Gitis**...

Cet article est publié avec l'aimable autorisation de **Christophe Porteneuve**, l'article original peut être lu sur le blog **JS Attitude : Enfin Maîtriser Les Expressions Rationnelles**.

I - À propos de l'auteur.....	3
II - Le croque-mitaine.....	3
III - Le bon outil pour manipuler du texte.....	3
IV - Un mot de vocabulaire.....	4
IV-A - Expression rationnelle ou régulière ?.....	4
IV-B - « reguex » ou « redjex » ?.....	4
V - Les exemples classiques.....	4
VI - Déconstruire une expression rationnelle.....	5
VII - Construire une expression rationnelle.....	5
VII-A - Apprivoiser les expressions rationnelles pas à pas.....	5
VII-B - Syntaxe des regex littérales en JavaScript.....	5
VII-B-1 - Délimiteurs.....	6
VII-B-2 - Drapeaux (flags).....	6
VII-C - Représenter un seul caractère de correspondance.....	6
VII-C-1 - Caractères littéraux (non spéciaux).....	6
VII-C-2 - Classes explicites.....	6
VII-C-3 - Classes prédéfinies.....	7
VII-C-4 - ASCII vs. Unicode.....	7
VII-C-5 - Combinaisons de classes.....	7
VII-C-6 - La classe générique : point.....	8
VII-D - Représenter plusieurs caractères successifs.....	8
VII-D-1 - Parties fixes.....	8
VII-D-2 - Quantificateurs.....	8
VII-D-3 - Quantificateurs réticents.....	9
VII-E - Alternative entre deux séquences.....	9
VII-F - Groupes.....	9
VII-F-1 - Grouper pour quantifier une séquence de caractères.....	9
VII-F-2 - Grouper pour limiter une alternative.....	10
VII-F-3 - Grouper pour récupérer individuellement des fragments précis de la correspondance.....	10
VII-F-4 - Numérotation et imbrication de groupes.....	10
VII-F-5 - Références arrières (« backrefs »).....	11
VII-F-6 - Groupes non capturants.....	11
VII-G - Indiquer la position avec les ancres.....	12
VII-H - Conditions sur la suite du texte (lookaheads).....	12
VII-H-1 - Lookaheads positifs.....	12
VII-H-2 - Lookaheads négatifs.....	12
VII-H-3 - Lookbehinds ?.....	13
VIII - Les API JavaScript qui utilisent les regex.....	13
VIII-A - String#match et String#replace.....	13
VIII-B - String#split.....	14
VIII-C - String#search.....	14
VIII-D - RegExp.....	14
IX - Mise en application.....	15
IX-A - Exercice 1.....	15
IX-B - Exercice 2.....	15
IX-C - Exercice 3.....	15
X - Étendre les regex JavaScript : XRegExp.....	16
X-A - Comment exploiter XRegExp ?.....	16
X-B - Groupes capturants nommés.....	16
X-C - Nouveaux drapeaux.....	17
X-D - Add-ons et prise en charge d'Unicode.....	18
XI - Quand ne pas utiliser les expressions rationnelles ?.....	19
XII - Quelques outils pour aider à la construction.....	20
Pour conclure.....	20
XIV - Envie d'en apprendre davantage ?.....	20
XV - Remerciements.....	20

I - À propos de l'auteur

Christophe Porteneuve a plus de 15 ans d'expérience professionnelle en développement Web ; ses premières applications Web avec une utilisation intensive de JavaScript remontent à 1997.

Auteur d'ouvrages de référence tant en France (**Bien Développer pour le Web 2.0** chez Eyrolles) que dans le reste du monde (**Prototype and script.aculo.us** et **Pragmatic Guide to JavaScript**, chez Pragmatic Programmers), il participe régulièrement à des conférences internationales de premier plan autour de JavaScript (The Ajax Experience, JSConf Europe).

Depuis 1996, Christophe a assuré plus de 7 000 heures de formation à destination des professionnels comme d'étudiants en écoles d'ingénieurs privées.

Contributeur noyau aux célèbres projets open-source Prototype, script.aculo.us et Ruby on Rails, membre de Paris Web depuis 2007 (et président de 2008 à 2009), il est aujourd'hui cofondateur et directeur technique de Delicious Insights, qui propose des formations de qualité sur **JavaScript** et **Git**.

II - Le croque-mitaine

Traditionnellement, les expressions rationnelles ne sont pas enseignées. Il est déjà assez difficile de trouver un prof de BTS ou DUT capable de faire véritablement du Java, sans même parler de Python, Ruby ou JavaScript, pour espérer avoir carrément des cours décents d'expressions rationnelles.

Popularisées par Perl, les expressions rationnelles débarquent en général au travers d'un morceau de code parfaitement abscons, comme le dégoût inattendu d'un fragment de fichier binaire au beau milieu du code source. Et de fait, quel développeur, pas forcément junior mais globalement sain d'esprit, n'aurait pas le cœur au bord des lèvres en tombant tout à coup sur ce genre de chose :

```
if (expr.match(/w+([([w-]+) (?:[*~^$]?=(['"])?(.[+?]\1\/)/))
```

C'est vrai, quoi, comprenez-les. Les pauvres, personne ne les a prévenus, sinon ils n'auraient pas choisi le pâté de tripes à midi, vous pensez bien, c'était trop risqué.

Faute d'enseignement, faute de rationalité justement, faute de démystification utile, les expressions rationnelles restent à la périphérie de nos regards et de notre conscience, les *banshees* du développement, on les garde soigneusement à distance de peur qu'en y touchant elles ne nous attrapent tout entiers et nous volent ce qui nous reste de raison, tels de petits Cthulhus tout de symboles vêtus.

III - Le bon outil pour manipuler du texte

Naturellement, il n'en est rien, ne prêtez pas attention à cette porte qui claque quelque part dans l'asile d'Arkham. Les regex restent un outil extrêmement efficace pour extraire rapidement des éléments spécifiques d'un contenu textuel. Par exemple, repérer certaines balises dans du HTML ; trouver les lettres en double dans un texte ; capitaliser un texte (mettre ses initiales en majuscules et le reste en minuscules) ; détecter qu'un texte est bien une adresse e-mail (ou IP, ou URL...) valide ; etc.

Il est bien sûr possible d'écrire des algorithmes manuels pour tous ces usages, mais ceux-ci présentent deux inconvénients majeurs vis-à-vis des expressions rationnelles équivalentes :

- ils sont souvent largement plus verbeux (jusqu'à plusieurs centaines de lignes de code) et donc sujets à potentiellement bien davantage de bogues, notamment des erreurs de bornes (les erreurs +1/-1, ou *fencepost errors*) et de conditions de boucle ;
- ils sont en général significativement plus lents que leur équivalent à base d'expressions rationnelles.

Vous me rétorquerez peut-être qu'ils sont bien plus lisibles. C'est un argument aussi classique que fallacieux.

À part les cas triviaux d'analyse de texte (par exemple, en compter les consonnes), ce n'est pas vrai : tout traitement un tant soit peu avancé requiert automatiquement un volume non négligeable de code, avec ses variables, conditions, etc. La lisibilité se dégrade rapidement, ou à défaut, on obtient du code lisible, mais volumineux, dont il faut en outre comprendre l'algorithme pour en saisir l'objectif. Même si on se cantonne aux cas où le code alternatif est court, l'expression rationnelle équivalente est, elle, triviale et ne devrait donc pas poser de souci de lisibilité.

Qui plus est, la lisibilité d'une expression rationnelle est perçue traditionnellement comme mauvaise pour deux raisons : d'une part, nombre de gens écrivent des expressions balourdes, pataudes, ne tirant pas le meilleur parti de la syntaxe qu'elles alourdissent inutilement. D'autre part, cette syntaxe, bien que grammaticalement assez simple, est effectivement assez peu intuitive : la majorité des développeurs ont donc connu le syndrome du « WTF?! » lors de leurs premières rencontres avec de telles expressions, ce qui entraîne un préjugé.

Enfin, nombre de personnes n'utilisent pas (où n'ont pas à disposition, comme c'est hélas le cas en JavaScript natif) le drapeau de syntaxe `x`, qui permet de découper et commenter une expression rationnelle lors de sa déclaration, ce qui accomplit des miracles en termes de lisibilité. Nous y reviendrons en fin d'article avec la bibliothèque XRegExp.

IV - Un mot de vocabulaire

IV-A - Expression rationnelle ou régulière ?

La terminologie anglaise ne connaît que *regular expression*, l'adjectif insistant sur le fait que, comme pour l'algèbre relationnelle, les expressions régulières sont basées sur des principes mathématiques déterministes et que la syntaxe est donc sans ambiguïté (si si !). En France, sans doute à la faveur de traductions initiales ayant pris quelques libertés, on s'est vite retrouvé avec les deux locutions **expression régulière** et **expression rationnelle**. On rencontre l'une à peu près aussi souvent que l'autre. J'ai personnellement pris le parti de la seconde, car je trouve que le **rationnelle** met davantage en avant ce principe de conception ; employez l'un ou l'autre comme bon vous semble, mais soyez juste cohérent avec vous-même !

Dans les deux cas, c'est tout de même assez long, aussi utilise-t-on la majeure partie du temps l'abréviation **regex**, en anglais comme en français, qui a en outre le mérite d'éviter le débat précédent. Dans le reste de cet article, j'emploierai cette abréviation dans un souci de concision.

IV-B - « reguex » ou « redjex » ?

C'est la question suivante, posée par ceux qui se soucient de leur prononciation (et je les en félicite). Bien que l'origine du *reg* (*regular*) incite à l'utilisation d'un g dur (« reguex »), la prononciation anglaise usuelle ne s'intéresse qu'à l'abréviation elle-même et comme le g est doux devant les e et les i, on prononce effectivement « redjex ».

V - Les exemples classiques

Afin d'essayer de bien illustrer tout l'intérêt que peut représenter l'emploi des regex dans son code, en voici quelques-unes assez classiques. Le lecteur qui pinaille trouvera naturellement tout un tas de contre-exemples, mais le but n'est pas ici de recenser les solutions en béton armé, qui doivent gérer une telle pléthore de cas qu'elles sont en effet complexes (bien moins que les algorithmes associés, néanmoins). Il s'agit de montrer des cas pas trop ahurissants ! On n'aura par ailleurs recours qu'aux syntaxes gérées nativement par JavaScript (pas de groupes nommés, notamment).

- `[0-9a-f]+` # Nombre entier hexadécimal.
- `(["']).*?\1` # Chaîne de caractères (guillemets simples ou doubles, mais cohérents entre eux ; pas d'échappement).
- `[w.-]+@[w-]+\.[w]{3,6}` # E-mail (basique).
- `[w-]+(\.[w-]+){1,}` # FQDN (nom de domaine, hors IRI).

- `<[>]+>` # Balise XML (ouvrante ou fermante).
- `\&(lt|gt|apos|quot|amp);` # Échappement XML (les cinq entités obligatoires).
- `(?:\d{1,3}\.){3}\d{1,3}` # IP (basique).
- `(?:(?:\d{1,2}|2[0-4]\d|25[0-5])\.){3}(?:\d{1,2}|2[0-4]\d|25[0-5])` # IP (exact : contrôle des valeurs).
- `[a-z]+://[^/]+(?:\d+)?(?:/[^\#]+)?(?:[#.]+)?` # URL (syntaxes des composants un peu laxistes...).

VI - Déconstruire une expression rationnelle

Devant le foutoir (ben si, des fois, le foutoir !) que peut représenter une regex au premier abord, l'important est de savoir la déconstruire. Et là, c'est un peu comme les poupées russes : on procède de l'extérieur vers l'intérieur. La clé est aussi de savoir bien repérer les classes, groupes et leurs éventuelles imbrications, pour délimiter ainsi notre analyse.

Par exemple, prenons la première, `[0-9a-f]+`. On y repère une classe, c'est-à-dire une série de possibilités pour un unique caractère : `[0-9a-f]`, qui est assez facile à décoder : de 0 à 9 et de a à f, donc un caractère hexadécimal. Et le quantificateur `+` vient indiquer qu'on s'attend à trouver ça au moins une fois.

Plus compliqué : `(?:\d{1,3}\.){3}\d{1,3}`. Là, on doit repérer tout de suite le groupe (non capturant en l'occurrence, mais passons) `(?:\d{1,3}\.)`, et noter qu'il est quantifié 3 fois. On va même remarquer rapidement que la suite de l'expression est en fait une redite du contenu du groupe, au point final près. Les regex n'ont hélas pas de syntaxe pour dire « schéma X, N fois, séparé par schéma Y » : on se retrouve le plus souvent à dire « X + Y, N fois, puis X une dernière fois ». Sans pouvoir nommer les groupes, ça fait du doublon, comme ici. Et le contenu répété, `\d{1,3}`, est assez simple : « un chiffre décimal, 1 à 3 fois ».

Et ainsi de suite, en découpant le problème originel de plus en plus, pour se concentrer sur des fragments assez petits pour être analysés directement sans péter un câble. Mais croyez-moi, avec un peu de pratique, c'est *beaucoup* plus facile qu'il n'y paraît.

VII - Construire une expression rationnelle

Pour construire une regex, on fait l'inverse : on part des petits composants, et on... compose, au fur et à mesure. Mieux on aura découpé à la base, plus la construction se fera naturellement, et moins on aura de répétition.

Supposons par exemple que vous souhaitiez créer une expression de validation d'un login, qui aurait les contraintes suivantes : uniquement des caractères alphanumériques ASCII, à raison d'un minimum de 6 et d'un maximum de 12. Comment s'y prend-on ?

Déjà, tous les caractères ont la même contrainte, ce qui est bon signe : ça va nous éviter de multiplier les composants. On doit donc d'abord exprimer la contrainte sur *un* caractère, sous forme d'une série de valeurs possibles. C'est là le rôle des classes, on écrit donc `[a-z0-9]` pour lister les caractères et chiffres ASCII. Il ne nous reste plus qu'à quantifier ça, entre 6 et 12 occurrences : `[a-z0-9]{6,12}`.

VII-A - Apprivoiser les expressions rationnelles pas à pas

La syntaxe des expressions rationnelles a finalement assez peu d'éléments potentiels. Et ce d'autant plus que JavaScript ne gère pas nativement certaines extensions intéressantes, comme le drapeau `x` ou les groupes nommés. Nous allons examiner ces possibilités tranquillement, une par une, dans la suite de cet article.

VII-B - Syntaxe des regex littérales en JavaScript

En JavaScript, vous avez deux manières d'obtenir une regex utilisable : en l'écrivant directement dans le code, de façon statique, sous forme d'un littéral. Ou en utilisant une construction d'objet explicite, avec la syntaxe `new RegExp`.

Dans la pratique, cette deuxième syntaxe n'a d'intérêt que si vous devez construire une regex dynamiquement sur la base de contenus que vous ne connaissez pas encore en écrivant le code (par exemple, une saisie utilisateur). Dans tous les autres cas, préférez la syntaxe littérale, plus concise.

Une regex littérale en JavaScript, ce sont trois choses : les délimiteurs, l'expression elle-même et les drapeaux de contrôle.

VII-B-1 - Délimiteurs

En JavaScript, on n'a pas le choix : une regex littérale est forcément encadrée par des *slashes* (`/`). Du coup, tout *slash* dans l'expression devra être échappé (`\`), ce qui peut vite donner des trucs cocasses (`/^https?:\V/`)...

VII-B-2 - Drapeaux (flags)

Les drapeaux de contrôle se placent après le *slash* de terminaison. JavaScript en reconnaît trois :

- `i` est très pratique et fréquent : *Insensible à la casse*. Permet d'éviter la galère de devoir spécifier tous nos caractères possibles en minuscules ET en majuscules, lorsqu'on accepte les deux...
- `g` rend la regex *Globale* : au lieu de se concentrer sur la première correspondance du texte, elle pourra agir sur toutes les correspondances. Change fondamentalement les comportements de recherche et de remplacement...
- `m` est souvent mal compris. Lui qui signifie *Multi-lignes* ne veut pas dire que notre regex va automatiquement marcher sur des correspondances à plusieurs lignes (et tout particulièrement, il ne permet pas au caractère générique total, `.`, de sauter les lignes), juste que les ancres (indications de position) `^` et `$` correspondront aux bords de ligne et non aux bords du texte total. Ce drapeau sert plus rarement.

Ainsi par exemple, une regex de nombre hexadécimal sera plus probablement `/[0-9a-f]/i`, car dans la plupart des langages, la casse des lettres est sans importance pour les littéraux numériques.

VII-C - Représenter un seul caractère de correspondance

Armés de ces connaissances de base sur la structure extérieure d'une regex, voyons l'intérieur. Une regex, fondamentalement, sert à chercher une correspondance dans un texte (voire à vérifier la correspondance du texte dans son ensemble). L'unité de base de ce travail, c'est le caractère. Quelles contraintes peut-on donc exprimer sur un seul caractère, pierre angulaire de la suite ?

VII-C-1 - Caractères littéraux (non spéciaux)

Tout d'abord, on peut exiger qu'il s'agisse d'un caractère précis. Lui et pas un autre. Il suffit alors d'utiliser le caractère directement, comme un littéral. Par exemple, `a` ne peut correspondre qu'au caractère A minuscule latin (code numérique 97).

Si votre caractère joue un rôle spécial dans les regex, il faudra l'échapper (sauf si vous êtes au sein d'une classe, ce qui la plupart du temps vous en dispense automatiquement, sympa). Par exemple, `/?[a-z]=/i` ne va pas faire correspondre un texte avec un point d'interrogation suivi d'une lettre ASCII suivi du signe égal : le point d'interrogation est un quantificateur, comme on le verra plus loin et s'attend donc même à avoir un contenu qui le précède : cette regex est invalide et votre interpréteur JS considérera ce morceau de code comme invalide. Il faudra employer `/\?[a-z]=/i` pour obtenir le résultat souhaité.

VII-C-2 - Classes explicites

Si on se contentait de comparer des caractères précis, on n'aurait pas besoin des regex : une bête comparaison de chaîne suffirait !

Afin d'exprimer une *série de possibilités* pour un caractère unique dans le texte analysé, on utilise des *classes*. Elles consistent à lister les caractères possibles, entre deux crochets. Par exemple, pour représenter les voyelles minuscules latines :

```
[aeiouy]
```

Évidemment, si on doit se farcir *toutes* les possibilités manuellement, ça va vite faire longuet... Lorsque les caractères possibles sont adjacents dans la table de caractères, on peut donc juste séparer les limites de la plage voulue par un tiret :

```
[a-z]
```

Et on peut naturellement combiner caractères explicites et plages, comme ci-dessous pour les chiffres hexadécimaux par exemple :

```
[a-f0-9]
```

Il peut par ailleurs arriver qu'on souhaite exprimer non pas tous les caractères autorisés mais plutôt, par concision, *tous les caractères interdits*. On parle alors de *classe négative*, qui doit commencer par un circonflexe. Ainsi, pour interdire les chiffres arabes par exemple, mais autoriser tout le reste, on dirait ceci :

```
[^0-9]
```

Le sens spécial qu'ont le crochet fermant, le tiret et le circonflexe à l'intérieur d'une définition de classe obligent soit à les échapper (en les précédant d'un *backslash*), soit, pour le tiret, à le placer en bord de classe, afin de lever l'ambiguïté (s'il n'a pas de caractère des deux côtés, il n'indique pas un intervalle).

VII-C-3 - Classes prédéfinies

Certaines séries sont suffisamment communes pour disposer de syntaxes raccourcies. Celles-ci se présentent sous forme d'un *backslash* suivi d'une lettre. Les trois lettres gérées par JavaScript sont :

- **\d** pour *digit*, qui regroupe les chiffres arabes ;
- **\w** pour *word*, qui regroupe les lettres latines ASCII, les chiffres arabes et le tiret de soulignement (*underscore*) ;
- **\s** pour *space*, qui regroupe les espacements usuels ASCII (espace, tabulations, saut de ligne, saut de page).

Si la lettre est minuscule, on a une classe positive (seuls ces caractères-là). Si elle est majuscule, on a une classe négative (tout sauf ces caractères-là). Ainsi, **\d** équivaut à `[0-9]` et **\W** équivaut à `[^a-z0-9_]`.

VII-C-4 - ASCII vs. Unicode

Il est dommage que JavaScript ne gère pas mieux les jeux de caractères « réels », ceux employés par les langues. Ainsi, **\w** ignore les signes diacritiques (accents, cédilles, etc.) et d'une manière générale toutes les lettres non latines (grecques, allemandes, asiatiques...). Dans le même esprit, **\s** ignore l'espace insécable, aussi fréquente soit-elle.

Nous verrons en fin d'article qu'une petite bibliothèque JavaScript, XRegExp, étend considérablement les possibilités des regex en JavaScript, si vous avez ce type de besoin.

VII-C-5 - Combinaisons de classes

Il est possible de combiner des classes prédéfinies au sein d'une classe explicite, éventuellement avec d'autres caractères spécifiques, par exemple comme ceci :

```
[ \wéèéèâââîîôöûîç\s\u00a0]
```

On trouve ici les alphanumériques latin/arabe, le soulignement, les principales lettres françaises à signe diacritique, les espacements ASCII et l'espace insécable. JavaScript nous autorise en effet à utiliser la syntaxe de code littéral Unicode `\uxxxx`. On a ici le code 160 (`a0` en hexadécimal), qui est l'espace insécable dans la table Unicode. C'est une syntaxe qu'on retrouve également dans CSS.

VII-C-6 - La classe générique : point

Une classe particulière est le *Any char* : `.` (point). Elle représente en théorie n'importe quel caractère. Dans la pratique, elle est limitée par l'activation (ou pas) d'un drapeau de comportement indiquant si elle comprend aussi les sauts de ligne ou non. Un tel drapeau n'existe pas dans les regex JavaScript standard et le point ne correspond jamais à un saut de ligne, même avec le mode multiligne (contrairement à d'autres moteurs de regex, comme par exemple celui de Ruby). Là encore, XRegExp offre une amélioration.

En tous les cas, il faut faire attention au risque d'employer un point littéral sans réfléchir, car le point des regex voudra dire « n'importe quel caractère » ! Pensez alors à l'échapper avec un *backslash*.

VII-D - Représenter plusieurs caractères successifs

Jusqu'à présent, nous nous sommes concentrés sur l'établissement d'un critère pour *un* caractère. Mais dans une analyse de texte, c'est une séquence de caractères que nous allons examiner !

VII-D-1 - Parties fixes

Le plus simple consiste à utiliser, à la suite, les descriptifs de motif pour chaque caractère du texte à analyser. Par exemple, si vous voulez juste une lettre suivie d'un chiffre, vous direz sans doute :

```
[a-z]\d
```

Toutefois, la plupart du temps vous souhaitez aussi indiquer qu'un motif donné est acceptable pour davantage qu'un seul caractère : peut-être 2, 10, au moins 20, maximum 15, un ou plus, ou un nombre quelconque (y compris aucun) ... C'est le rôle des *quantificateurs*.

VII-D-2 - Quantificateurs

Un quantificateur est une syntaxe spéciale qui, employée derrière un élément de la regex, modifie sa quantité potentielle. À la base, tout composant de la regex est attendu exactement une fois. On peut toutefois le faire suivre des syntaxes de quantification que voici :

- `?` signifie « zéro ou un », ce qui revient à dire « optionnel » ;
- `+` signifie « au moins un » ;
- `*` signifie « un nombre quelconque de fois » (y compris zéro).

Il est aussi possible d'utiliser des bornes précises, au moyen d'une syntaxe un peu plus longue, encadrée par des accolades (*curly braces*) :

- `{4}` pour « exactement 4 fois » ;
- `{4,12}` pour « entre 4 et 12 fois » ;
- `{4,}` pour « au moins 4 fois ».

La syntaxe `{,4}` n'existe pas, pour la simple raison qu'on obtient le même résultat avec `{0,4}`.

VII-D-3 - Quantificateurs réticents

Les quantificateurs vus à l'instant sont tous qualifiés de *gourmands* (*greedy quantifiers*), au sens où ils tenteront systématiquement d'obtenir *la plus grande correspondance possible*. Ce n'est pas toujours ce qu'on veut. Prenons par exemple le texte analysé suivant :

```
<p><a href="#deuze">Et yop la deuze</a> - <a href="#quarte">Et yop la quarte</a></p>
```

Imaginons maintenant que nous voulons récupérer les liens. Une regex naïve serait `<a.*>.+</a>/g`. Mais ça va nous donner un unique résultat : `"<a href="#deuze">Et yop la deuze</a> - <a href="#quarte">Et yop la quarte</a>"`. Ah. Flûte. Le souci ici vient du fait qu'on utilise des quantificateurs gourmands. On pourrait bien sûr remplacer le premier par `[^>]*` histoire d'être sûrs de s'arrêter en fin de balise ouvrante, mais ça ne marche pas pour le deuxième, car si on faisait `...>[^<]+</a>`, on s'arrêterait dès qu'un lien contient des balises...

On peut plutôt recourir à des versions réticentes.

Il suffit de rajouter un `?` à la fin du quantificateur gourmand usuel :

```
/<a.*?>.+?</a>/g
```

Et là, on obtient bien deux résultats, un par lien. Les quantificateurs vont s'arrêter au *premier chevron fermant rencontré* et au *premier rencontré*.

VII-E - Alternative entre deux séquences

Pour indiquer une alternative entre caractères, on utilise généralement une classe. Par exemple, `[arz]` signifie « a, ou r, ou z ». Mais comment faire pour des séquences ? On les sépare par un *pipe* (`|`).

Ainsi, `hello|world` signifie « hello ou world ». Et `[1-9][0-9]*|0x[0-9a-f]+` signifie « un littéral nombre décimal (chiffre de 1 à 9 suivi éventuellement de chiffres décimaux) ou un littéral nombre hexadécimal (0 puis x puis un nombre quelconque non nul de chiffres hexadécimaux) ».

Attention cependant, il peut y avoir un piège quant à la portée de l'alternative : par défaut, elle s'applique à *tout ce qui l'entoure* ! C'est vite un souci, mais nous allons voir comment le résoudre dans un instant.

VII-F - Groupes

Il est possible de rassembler des fragments de vos regex en groupes. Il y a plusieurs raisons de faire cela : pour quantifier plus qu'un simple caractère (mais carrément toute une séquence), pour limiter une alternative, ou pour obtenir comme résultat, en plus de la correspondance globale, des segments précis de celle-ci.

La syntaxe fondamentale des groupes est toujours la même : on entoure un groupe par des parenthèses : `( )`.

VII-F-1 - Grouper pour quantifier une séquence de caractères

Imaginons que vous souhaitiez faire une regex toute bête : « au moins deux mots ». Au sens « au moins deux séries de non-espacements », par exemple. La séquence « série de non-espacements » est assez facile : `\S+` (majuscule). Et les mots seraient séparés par, justement, des séries d'espacements : `\s+` (minuscule).

Pour dire « deux mots », on pourrait donc faire `\S+\s+\S+`. Hmmmm. Mais pour dire « au moins deux mots » ? On fait comment. Comme souvent lorsqu'il s'agit de séquences alternées (motifs X séparés par motif Y), on doit la représenter en deux parties :

- 1 Motif X.
- 2 Motif Y puis Motif X, le tout un certain nombre de fois (le tout quantifié, quoi).

Ici, vu qu'on veut « au moins deux mots », le quantificateur de cette deuxième partie serait « au moins une fois », donc le `+`. Mais on ne peut pas juste faire `\S+\s+\S++`, ça n'aurait aucun sens. Comment faire que ce dernier `+` sache à quoi il s'applique, en l'occurrence `\s+\S+` ? Avec les groupes, pardi :

```
\S+(\s+\S+)+
```

Le groupe constitue une seule entité grammaticale, donc le quantificateur qui le suit s'applique au groupe dans son entier.

VII-F-2 - Grouper pour limiter une alternative

Un autre emploi fréquent du groupe est pour limiter une alternative (une utilisation de `|`, le *pipe*). Par défaut, on l'a vu, une alternative s'applique à tout ce qui l'entoure. Mais si elle figure au sein d'un groupe, elle se limite à ce groupe.

Supposons que vous vouliez autoriser les textes « hello world », « hallo world » et « hi world ». Le « world » étant commun on peut limiter l'alternative au premier mot. Mais si on fait ceci :

```
hello|hallo|hi world
```

...on se plante, car ça correspondrait à « hello », « hallo » ou « hi world », ce qui n'est pas ce qu'on veut. Il faut délimiter l'alternative avec un groupe :

```
(hello|hallo|hi) world
```

Et voilà, le tour est joué !

VII-F-3 - Grouper pour récupérer individuellement des fragments précis de la correspondance

Le dernier usage des groupes, qui est extrêmement utile, est la possibilité de référencer des portions spécifiques de la correspondance obtenue. Par exemple, vous essayez de repérer, au sein d'un texte, une chaîne française de date, au format `jj/mm/aaaa`. Non seulement vous voulez vérifier que la chaîne est bonne, mais aussi en extraire les composants. Au lieu de « juste » écrire ceci :

```
\d{2}/\d{2}/\d{4}
```

...vous pourriez isoler chaque composant dans son groupe :

```
(\d{2})/(\d{2})/(\d{4})
```

Vous allez alors obtenir, en plus de la correspondance globale (par exemple « 25/07/2012 ») des groupes 1 à 3 (« 25 », « 07 » et « 2012 »).

Les groupes sont numérotés de 1 à 9. Au-delà, vous ne disposez plus de numéros. Vous ne pouvez donc avoir recours « que » à 9 groupes numérotés dans une expression. On parle de *groupes capturants*.

VII-F-4 - Numérotation et imbrication de groupes

Il est naturellement possible d'imbriquer les groupes. Par exemple, comme ceci :

```
(\d{4})-(\d{2})-(\d{2})
```

Mais alors, quel groupe a quel numéro, et quelle valeur ? La règle de numérotation est simple : *dans l'ordre des parenthèses ouvrantes*. Ainsi, sur le texte 2012-07-28, on obtiendrait :

- le groupe 1 : 2012-07-28 ;
- le groupe 2 : 07 ;
- et le groupe 3 : 28.

VII-F-5 - Références arrières (« backrefs »)

Dès l'instant où un groupe est numéroté, il devient possible de le référencer, tant pour un remplacement que pour une recherche.

Le cas de figure du remplacement est assez classique et utilise la syntaxe `$numéro`. Supposons que vous ayez à transformer des dates américaine (mois/jour/année) en dates techniques (année-mois-jour). On pourrait procéder ainsi :

```
usDate.replace(/(\d{2})\/(\d{2})\/(\d{4})/, '$3-$2-$1')
```

Là où ça devient vraiment pratique, c'est au sein même du motif de recherche. On s'en sert couramment pour faire correspondre des délimiteurs. Supposons que vous vouliez définir un motif qui autorise de part et d'autre l'apostrophe (') ou le guillemet ("), mais qui exige que les deux extrémités correspondent. La version pourrie, ce serait :

```
'\.+?'|".+?'
```

Et encore, elle pose plein de soucis. Mais vous pouvez facilement simplifier ça (surtout si ce qu'il y a entre les apostrophes/guillemets est un motif compliqué) en évitant toute répétition. Pour le cas où l'apostrophe/guillemet initial constituerait le premier groupe numéroté du motif :

```
(['"])\.+?\1
```

Pratique, non ? Le `\1` fait référence au texte qui *a correspondu au groupe 1* : il faut alors que dans le texte analysé, on trouve à l'endroit du `\1` *exactement le même texte que pour le groupe 1*.

VII-F-6 - Groupes non capturants

Neuf groupes, c'est beaucoup et vous n'aurez normalement pas besoin de tous. En revanche, il arrive souvent que vous ne *souhaitiez pas exploiter le groupe individuellement*, et n'ayez donc pas besoin de lui attribuer un numéro. Par exemple, vous l'auriez utilisé uniquement pour une des autres raisons : quantifier une séquence ou limiter une alternative. Il est alors dommage de « polluer » les groupes numérotés légitimes avec ceux-là, vous obligeant à examiner les groupes 2, 6 et 9 au lieu de 1, 2 et 3. Comment faire ?

C'est tout simple, il suffit de recourir alors à des *groupes non capturants*. Par défaut, un groupe est *capturant* : il s'octroie un numéro. Mais lorsque vous l'utilisez juste à des fins structurelles, vous pouvez le rendre non capturant en préfixant son contenu par `?`. Ça donnerait quelque chose comme ça :

```
([A-Z]{2})(?:\.\d+)([A-Z])
```

Dans l'exemple ci-dessus, on a d'abord un groupe capturant, qui comprend deux lettres latines majuscules : `([A-Z]{2})`. C'est le groupe numéro 1. Puis on a un groupe non capturant, dont le seul objectif est de pouvoir ensuite appliquer à son contenu le quantificateur `+` : `(?:\.\d+)`, à savoir un point suivi d'au moins un chiffre. Et finalement, on a un deuxième groupe capturant, qui aura donc le numéro 2 : `([A-Z])`, soit une lettre latine majuscule.

En utilisant judicieusement la syntaxe non capturante, vous n'utilisez des numéros que pour les groupes que vous souhaitez effectivement référencer ensuite. En plus, un groupe capturant est souvent plus lourd à gérer par les

moteurs de regex qu'un groupe non capturant : vous améliorez donc aussi, même si ce n'est pas toujours sensible, la performance de votre regex.

VII-G - Indiquer la position avec les ancres

Jusqu'ici, nous avons rédigé des regex qui, dans la pratique, pouvaient correspondre à un texte figurant n'importe où dans la chaîne de caractères examinée.

Par exemple, la regex `h[ea]ll?o` correspondra bien sûr à `hallo`, `hello` et `halo`, mais aussi à `hello world`, `say hello`, `mark!` et surtout à `chaloupe` ou `halogène`.

Il arrive fréquemment qu'on ait besoin de préciser une contrainte de positionnement sur tout ou partie d'une regex. C'est le rôle des *ancres*.

Les regex JavaScript reconnaissent les ancres suivantes :

- `^` (circonflexe, en anglais *caret*). Il s'agit du début de la chaîne de caractère examinée. Si la regex utilise le drapeau multilignes (`m`), il peut en fait s'agir de n'importe quel début de ligne (début de texte ou juste après un saut de ligne, `\n`) ;
- `$` (dollar). C'est la fin de la chaîne, ou en mode multiligne ce peut aussi être une fin de ligne ;
- `\b` est le *word Boundary*. C'est une ancre extrêmement utile, qui désigne une *bordure de mot*. Ça inclut notamment les sauts de lignes, espaces, la ponctuation et les extrémités du texte. Par symétrie, on a aussi `\B` (majuscule), qui comme pour les classes est le contraire de `\b` : elle indique qu'on ne se *situe pas* à la bordure d'un mot.

Quelques exemples :

```
^halo // correspond à "halo" et "halogène" et "halo éclatant", mais pas à "chaloupe" ou "un halo."
halo$ // correspond à "halo", mais ni "halogène", ni "halo éclatant", ni "chaloupe", ni "un halo."
\bhalo // correspond à "halo", "un halo.", "halo éclatant" et "halogène", mais pas "chaloupe"
halo\b // correspond à "halo", "un halo." et "halo éclatant", mais ni "halogène" ni "chaloupe"
```

Ainsi, pour indiquer qu'une regex correspond à *l'ensemble d'un texte*, on l'encadre par `^` et `$`, comme ceci :

```
^h[ea]llo w(?:orld|elt)$ // "hello world", "hallo welt", "hello welt", "hallo world" et c'est tout.
```

VII-H - Conditions sur la suite du texte (lookaheads)

Parfois, une contrainte sur la position ne suffit pas. Il faut carrément indiquer une condition sur *la suite du texte*.

VII-H-1 - Lookaheads positifs

Peut-être quelque chose comme « un chiffre, à condition qu'il soit suivi par deux autres chiffres ». Mais on ne veut pas pour autant accumuler les deux chiffres qui suivent dans la correspondance. On utilise pour cela un *positive lookahead*, identifié comme un groupe avec un préfixe `?=` :

```
\d(?!=\d\d) // "2" dans "237" ou "243", mais pas dans "24M", "2", "2T2" ou "2T4".
```

VII-H-2 - Lookaheads négatifs

Il est évidemment possible aussi d'exprimer une contrainte *négative* sur la suite du texte : ce qu'on ne *veut pas* trouver après. Au lieu de `?=`, on utilise alors `?!`. Pour prendre l'inverse de l'exemple précédent, on pourrait dire « un chiffre, pourvu qu'il ne soit pas suivi par un autre chiffre » :

```
\d(?:\d) // "2" dans "2", "2TZ" ou "2T4", mais pas dans "237", "243" ou "24M".
```

Tout ça peut sembler un peu basique, mais voici un bel exemple combiné et imbriqué, et une fois que vous l'avez digéré, vous êtes au point sur les lookaheads ! Examinez plutôt :

```
numberString.replace(/(\d) (?=(\d\d\d)+ (?! \d))/g, "$1 ")
```

Oh pétard ! Ça fait quoi, ça ?!?

Eh bien « tout simplement », ça ajoute les espaces en séparateurs de milliers entre les groupes de trois chiffres (en partant de la droite, évidemment). En somme, grâce aux *lookaheads*, on arrive à analyser *par la droite* alors que les regex travaillent *par la gauche*.

Voilà de quoi vous faire tourner les méninges... Petite astuce pour décrypter cette regex : on cherche en fait à remplacer chaque *fin de groupe numérique* (hors le dernier) par son dernier chiffre suivi du séparateur. Et comment sait-on qu'on a des groupes numériques valables derrière soi ? Parce qu'on a *un nombre de chiffres multiple de 3* derrière soi. Voilà, je vous laisse décortiquer tout ça... Quelques exemples de conversion :

```
'243' // '243'
'2431' // '2 431'
'24315' // '24 315'
'243157' // '243 157'
'2431576' // '2 431 576'
...
```

VII-H-3 - Lookbehinds ?

Dans certains cas de figure, il peut être également utile de poser une contrainte sur *le texte d'avant*, indépendamment de comment on a éventuellement traversé ce texte dans un début de correspondance. On aurait alors recours aux *lookbehinds*, qui traditionnellement s'écrivent `?<=` et `?<!`. Cependant, JavaScript ne propose pas, en standard, cette syntaxe.

Et pour le coup, XRegExp ne permet pas de simuler cette possibilité. Il existe toutefois la possibilité de **les simuler si besoin**.

VIII - Les API JavaScript qui utilisent les regex

JavaScript nous permet de recourir aux regex au moyen de diverses API, certaines très connues, d'autres moins.

VIII-A - String#match et String#replace

Certainement les deux plus connues. On utilise la première pour chercher des correspondances, l'autre pour les remplacer.

Selon que la regex utilise le drapeau `g` ou non, on cherche (ou remplace) toutes les correspondances, ou seulement la première :

```
'hello world!'.match(/\w+/g) // => ['hello', 'world']
'hello world!'.match(/\w+/) // => ['hello']
```

Ce que beaucoup ne savent pas, c'est que le résultat n'est pas juste un tableau, avec sa propriété `length` et les index numériques (0, 1, etc.). En tout cas, quand le drapeau `g` est absent. Il est alors aussi doté de propriétés spéciales : `index`, qui indique la position de la première correspondance, et `input`, qui reprend le texte complet analysé. Voyez plutôt :

```
var result = 'hello world!'.match(/\s(\w+)/)
result.length // => 2
result[0]      // => ' world' (la correspondance complète)
result[1]      // => 'world' (le groupe numéroté 1)
result.index   // => 5 (la position, en partant de 0, de la correspondance complète dans le texte)
result.input   // => 'hello world!' (le texte analysé)
```

Quant à `replace`, elle accepte en fait deux formats pour son deuxième argument (qui décrit le remplacement).

Le premier format est une chaîne de caractères : c'est le motif de remplacement. On peut alors y trouver les syntaxes spéciales :

```
$& (la correspondance complète)
$` (le texte avant la correspondance)
$' (le texte après)
$1 à $9 (les groupes numérotés).
```

Par exemple :

```
'hello world!'.replace(/\w+/g, '[$&]') // => '[hello] [world]!'
```

On peut aussi, et c'est là que ça devient balèze, utiliser une fonction, qui prend alors comme arguments la correspondance totale, les groupes numérotés, l'index de début de correspondance et finalement le texte analysé d'origine. La fonction renvoie le texte qui remplacera la correspondance globale. Voyez plutôt :

```
'HELLO WORLD!'.replace(/\b(\w)(\w*)/g, function(word, initial, remainder) {
  return initial.toUpperCase() + remainder.toLowerCase();
})
// => 'Hello World!'
```

VIII-B - String#split

La plupart des développeurs connaissent `split`, par exemple comme ci-dessous :

```
'hello world this is nice'.split(' ') // => ['hello', 'world', 'this', 'is', 'nice']
```

Mais beaucoup ne savent pas, et c'est dommage, que le séparateur peut être une regex. Ainsi, imaginez le texte suivant :

```
'01-17.32 28--17/41'
```

Pour extraire les groupes numériques, avec un séparateur textuel, on est dans la mouise. Mais avec une regex, aucun souci :

```
'01-17.32 28--17/41'.split(/\D+/) // => ['01', '17', '32', '28', '17', '41']
```

VIII-C - String#search

Pour information, il existe aussi une API `search` qui vous renvoie uniquement la position de la correspondance (-1 si introuvable), et non tout le détail que renvoie `match`. Du coup, si c'est la seule info qui vous intéresse (la position), c'est plus performant.

VIII-D - RegExp

Les regex JavaScript sont toutes des objets (comme presque toujours en JavaScript), en l'occurrence des instances de `RegExp`. Elles ont notamment deux méthodes : `exec()` et `test()`.

Voici deux expressions rigoureusement équivalentes :

```
/\w+/.exec('hello world!') // <=> 'hello world!'.match(/\w+/)
```

Le drapeau `g` est sans effet sur `exec()`, en revanche. On tape toujours dans la première correspondance. Le résultat renvoyé est strictement identique à celui décrit pour `match()`, avec les propriétés numériques ainsi que `index` et `input`.

Si tout ce qui vous intéresse, c'est qu'il y ait ou non correspondance, vous pouvez vous contenter de `test()`, qui renvoie un simple booléen et s'exécute donc plus rapidement (un peu comme tout à l'heure, avec la différence entre `match()` et `search()`).

Il est donc possible de créer dynamiquement une regex, avec `new RegExp(motif, drapeaux)`. Le seul intérêt, naturellement, par rapport à un littéral, c'est si le contenu du motif (ou des drapeaux) varie avec le temps, par exemple suite à une saisie utilisateur.

Supposons que vous autorisiez l'internaute à taper des lettres qui servent de filtre de recherche, ce qu'on appelle une *fuzzy search* : ça correspondrait à tout texte où ces lettres existent dans l'ordre, peu importe ce qu'il y aurait entre elles.

Par exemple, pour la saisie `jel`, vous voudriez rechercher tout texte contenant un `j`, plus loin un `e` et encore plus loin un `l`. En somme, la regex serait `j.*e.*l`. Supposons que la saisie soit effectuée dans un champ dont l'attribut `id` vaudrait `filter` :

```
var expr = document.getElementById('filter').value; // $('#filter').val() avec jQuery...
var regex = new RegExp(expr.split('').join('.*'), 'gi'); // Peu importe la casse...
```

En réalité, pour que cet exemple soit vraiment blindé, il faudrait d'abord « échapper » les caractères spéciaux saisis (spéciaux au sens des regex), en les préfixant par un *backslash* (`\`), mais vous saisissez l'idée.

C'est bien là la seule raison de recourir à `new RegExp` (ainsi peut-être qu'un motif bourré de *slashes* (`/`)). Le reste du temps, un littéral est plus concis et plus performant.

IX - Mise en application

Afin de vous permettre de vous faire la dent, voici quelques exercices pour pratiquer.

IX-A - Exercice 1

Prenez le premier paragraphe de **ce lorem ipsum** et collez-le dans une `String`. Extrayez-en les mots de plus de quatre lettres non suivis d'un espace.

Voir la solution sur JSBin

IX-B - Exercice 2

Parmi les **acronymes connus commençant par AC**, sortez tout ceux comportant une répétition consécutive de lettres, quelles qu'elles soient.

Voir la solution sur JSBin

IX-C - Exercice 3

Examinez le HTML résultat de **cette recherche Google** et sortez les titres, URLs et positions (au sein des résultats naturels uniquement) des pages JS Attitude.

[Voir la solution sur JSBin](#)

X - Étendre les regex JavaScript : XRegExp

Bien que le moteur de regex de JavaScript ne soit pas mauvais (et plutôt rapide), à l'usage, on regrette rapidement l'absence de certaines fonctionnalités présentes dans des moteurs plus récents comme ceux qu'on peut trouver dans les versions actuelles de Perl, Python ou Ruby, pour ne citer qu'eux.

Heureusement, grâce au boulot acharné de Steve Levithan, fan de regex et de JavaScript, il existe un palliatif de premier choix : l'excellente bibliothèque **XRegExp**.

Celle-ci nous offre de nouvelles syntaxes, de nouveaux drapeaux de traitement et va même jusqu'à corriger quelques bogues exotiques des moteurs de regex proposés par les principaux navigateurs.

X-A - Comment exploiter XRegExp ?

XRegExp n'est pas natif à JavaScript, aussi les littéraux de type `/.../` ne peuvent pas s'en servir automatiquement. Il faut recourir à la fonction **XRegExp**, qui a les mêmes arguments que le constructeur `new RegExp` : le motif et les drapeaux. Par exemple :

```
var regex = XRegExp('<p class="result">.+?</p>', 'is');
```

Cette syntaxe a un léger inconvénient (évidemment) : puisqu'on est dans un littéral chaîne de caractères et non un littéral regex, il va falloir doubler tous les *backslashes*. Ainsi, `/\w+\s` donnera `XRegExp("\\w+\\s")`.

Là où les instances de `RegExp` et `String` ont des méthodes directes, **XRegExp** a choisi d'avoir une API 100 % « statique », callable dans l'espace de nom **XRegExp**. Ce ne sont pas des méthodes sur les objets retournés par `XRegExp(...)`. On trouve donc une méthode `XRegExp.exec` et une méthode `XRegExp.test`, mais il y a aussi les équivalents des méthodes complémentaires disponibles sur `String` : `split` et `replace`. Par ailleurs, l'API de **XRegExp** fournit de nombreuses méthodes utilitaires, comme `forEach` pour itérer facilement sur les correspondances, `escape` et `build` pour construire dynamiquement une regex depuis des fragments divers, et j'en passe. **Jetez un œil à l'API complète.**

X-B - Groupes capturants nommés

XRegExp nous fournit de nombreuses améliorations syntaxiques. Celle qui est de loin ma préférée consiste à permettre de nommer les captures. Ainsi, plutôt que d'avoir un bête numéro de 1 à 9, on a un nom explicite. Par exemple, au lieu de ceci :

```
/(\w+)\s+1/
```

...on peut préférer ceci (certes plus long, mais plus explicite) :

```
XRegExp('( ?<word>\\w+)\\s+\\k<word>')
```

Nous verrons des exemples plus « percutants » tout à l'heure, en combinant cette possibilité avec la syntaxe étendue.

Un groupe nommé est forcément capturant (il se voit donc aussi affecter un numéro, comme une capture usuelle) et démarre par le préfixe `?<nom>`. Une *backref* nommée, au lieu d'être *backslash* + numéro, est `\\k<nom>`.

Ce qui est vraiment sympa, c'est que cette prise en charge va jusqu'au bout :

- dans les textes de remplacement utilisés comme deuxième argument de `replace` : on a droit à la syntaxe `${nom}` ;
- dans les propriétés de l'objet résultat renvoyé par `match` et transmis aux fonctions de remplacement : en plus des propriétés numériques usuelles, on a des propriétés nommées comme le groupe.

```
var regex = XRegExp('(?(<word>\\w+)\\s+\\k<word>');
var text = 'Franchement XRegExp XRegExp ça déboîte !';
var result = XRegExp.exec(text, regex);
result.word // => 'XRegExp'
```

XRegExp fournit encore d'autres possibilités, telles que les commentaires monoligne intégrés ou les commutateurs de mode à la volée, mais je leur trouve peu d'intérêt. Vous pouvez en savoir plus sur la page des [syntaxes proposées par XRegExp](#).

X-C - Nouveaux drapeaux

XRegExp introduit par ailleurs trois nouveaux drapeaux qui altèrent le fonctionnement standard du moteur. En plus des drapeaux usuels (`i`, `m` et `g`), on trouve désormais :

- `s` pour *Singleline*. Ce mode est également connu sous le nom « dotall », et signifie que le caractère générique `.` (point) peut correspondre à véritablement n'importe quoi, y compris des sauts de ligne (quatre caractères Unicode possibles, dont `\n` et `\r`). Extrêmement pratique quand on traite du HTML et qu'on veut extraire des balises de contenu...
- `n` est le drapeau de *capture explicite*. Avec lui, seules les groupes nommés sont capturants : les groupes non nommés n'ont plus besoin d'être préfixés par `?` pour être non capturants ;
- `x` est le *eXtended mode*, votre nouveau meilleur ami. Il permet d'utiliser un *espacement libre* et aussi des *commentaires de fin de ligne*. Prenons l'exemple d'une analyse de date au format A-M-J : `(\d{4})-(\d{2})-(\d{2})`

Pas horrible, mais pas top non plus. On pourrait faire :

```
XRegExp('(\\d{4}) # Year \\
- \\
(\\d{2}) # Month \\
- \\
(\\d{2}) # Day')
```

Déjà plus lisible, non ? Grâce à la possibilité d'ignorer les espacements non qualifiés, tout espacement littéral est considéré comme ignorable dans le motif. À vous d'utiliser une autre syntaxe (le générique `s` ou une entité numérique spécifique) en cas de besoin.

On peut remplacer (ou augmenter) ces commentaires assez simples en nommant les captures :

```
XRegExp('(?(<year>\\d{4}) - (?(<month>\\d{2}) - (?(<day>\\d{2}))')
```

Du coup, l'objet résultat n'aurait pas seulement des propriétés 1, 2 et 3, mais aussi et surtout `year`, `month` et `day`.

Et maintenant une décomposition d'URL pas trop tatillonne :

```
var regex = XRegExp('^(?(<scheme> [^:/?]+ ) :// # aka protocol  \\n\\
(?(host> [^/?]+ ) # domain name/IP \\n\\
(?(path> [^?]* ) \\/? # optional path \\n\\
(?(query> .* ) # optional query', 'x');
var text = 'http://google.com/path/to/file?q=1';

var parts = XRegExp.exec(text, regex);
parts // => ['http://google.com/path/to/file?q=1', 'http', 'google.com', '/path/to/file', 'q=1']
parts.scheme // => 'http'
parts.host // => 'google.com'
```

```
parts.path // => '/path/to/file'
parts.query // => 'q=1'
```

X-D - Add-ons et prise en charge d'Unicode

Dans un Web en UTF-8, l'incapacité de `RegExp` à gérer Unicode, que ce soit pour les espaces, les lettres ou les chiffres, devient de plus en plus gênante. `XRegExp` propose des *add-ons* pour diverses fonctions, au premier rang desquelles la capacité à détecter des *catégories Unicode*. À noter que `XRegExp` gère déjà correctement les espaces Unicode (et notamment l'espace insécable, code 160) avec `\s` (donc `\s...`).

De telles catégories sont alors utilisables dans les motifs de correspondance avec la syntaxe `\p{X}` (où `X` est la lettre abrégée de la catégorie) ou `\p{Category}` (avec le nom complet).

- **Unicode Base** fournit la catégorie la plus critique : **Letter** (abréviation `L`).
- **Categories** fournit les catégories/propriétés Unicode, telles que *Lowercase Letter*, *Uppercase Letter*, *Number* ou *Punctuation*.
- **Scripts** fournit tous les scripts (langues) définis par l'*Unicode Multilingual Plane*, tels que *Latin*, *Arabic*, *Greek*, *Tamil*...
- **Blocks** fournit les blocs Unicode, sortes de sous-ensembles des scripts, tels que *InLatin_Extended_A* ou *InControl_Pictures*.
- **Properties** est un complément de *Categories*, nécessaire à la prise en charge d'Unicode dite « niveau 1 ». On y trouve des séries telles que *Alphabetic*, *WhiteSpace*, *Ascii* ou *Assigned*.

Le site officiel reprend notamment ces exemples :

```
// Categories
XRegExp('\p{Sc}\p{N}+'); // Sc: currency symbol, N: number

// Scripts
XRegExp('\p{Cyrillic}');
XRegExp('[\p{Latin}\p{Common}]');

// Blocks (use 'In' prefix)
XRegExp('\p{InLatinExtended-A}');
XRegExp('\p{InPrivateUseArea}'); // Uppercase \P for negation
XRegExp('\p{^InMongolian}'); // Alternate negation syntax

// Properties
XRegExp('\p{Assigned}');
```

Les add-ons sont à charger après le script de base. Le site officiel fournit une version totale déjà minifiée, pour ceux qui en ont souvent besoin et préfèrent réduire le nombre de fichiers.

```
<script src="xregexp.js"></script>
<script src="addons/unicode/unicode-base.js"></script>
<script>
  var unicodeWord = XRegExp("^\p{L}+$");

  unicodeWord.test("???????"); // true
  unicodeWord.test("???"); // true
  unicodeWord.test("???????"); // true
</script>

<!-- \p{L} is included in the base script, but other categories, scripts,
and blocks require token packages -->
<script src="addons/unicode/unicode-scripts.js"></script>
<script>
  XRegExp("^\p{Katakana}+$").test("????"); // true
</script>
```

Vous pouvez consulter la [liste complète des add-ons XRegExp](#).

XI - Quand ne pas utiliser les expressions rationnelles ?

On a principalement deux cas :

- le besoin est suffisamment simple pour être traité plus rapidement par des moyens textuels classiques ; comme un moteur de regex sera toujours plus lourd / plus lent qu'un traitement textuel basique, recourir à une regex serait alors inutilement bourrin ;
- le besoin est trop compliqué pour être traité de façon fiable par les regex ; la syntaxe des regex ne permet pas d'exprimer tout et n'importe quoi. Au-delà d'une certaine limite, on a besoin d'un moteur plus adapté à la nature du texte examiné, quitte à mélanger les approches entre moteur dédié, textuel basique et regex.

On serait dans le premier cas pour détecter, par exemple, la position/présence d'un texte **fixe** au sein d'une chaîne de caractères.

En revanche, tout traitement basé sur la *sémantique* du texte est pénible en regex. Celles-ci opèrent, fondamentalement, au niveau du caractère, pas de la sémantique d'ensemble. Quelques exemples classiques devraient vous aider à établir la distinction.

L'analyse du XML/HTML est, de façon générale, difficile en regex. On peut bien sûr procéder à quelques extractions simples, comme avec l'exercice de tout-à-l'heure sur les liens de résultats naturels chez Google. Mais corréler deux liens entre eux ? Déterminer si certains liens ont telle ou telle propriété ? Comptabiliser des caractéristiques ? C'est une autre paire de manches, qui requiert soit un complément d'analyse sur l'extraction produite, soit une traversée par un mécanisme plus dédié au problème (en l'occurrence le DOM, XPath et un algorithme à soi).

Prenons un autre exemple très simple : déterminer la validité d'une adresse IP classique (une adresse IPv4). On a besoin de vérifier au minimum deux éléments :

- 1 On a affaire à quatre nombres séparés par un point.
- 2 Chaque nombre est entre 0 et 255.

Le premier point est déjà un peu balourd en regex, qui n'a pas d'opérateur « séparé par », et nous obligerait à répéter un motif. Mais le deuxième point, qui se penche sur la *sémantique* du nombre, illustre toute la limite des regex. L'expression nécessaire montre à quel point on travaille au niveau du caractère :

```
[01]?[0-9]{1,2}|2[0-4][0-9]|25[0-5]
```

Ce qui donne tout de même, pour l'ensemble :

```
(?:[01]?[0-9]{1,2}|2[0-4][0-9]|25[0-5]\.){3}[01]?[0-9]{1,2}|2[0-4][0-9]|25[0-5]
```

Alors qu'ici, un traitement textuel classique est amplement suffisant. Voici une possibilité qui essaie d'être efficace (au sens où elle « répond » aussitôt que possible) :

```
function isIPv4(text) {
    var components = text.split('.');
    if (4 != components.length)
        return false;
    for (var index = 0; index < 4; ++index) {
        var comp = parseInt(components[index], 10);
        if (comp < 0 || comp > 255)
            return false;
    }
    return true;
}
```

Oui, c'est plus long. Mais c'est aussi plus rapide à analyser et comprendre pour quelqu'un qui ne passe pas ses journées dans les regex...

XII - Quelques outils pour aider à la construction

Lorsqu'on découvre et pratique les regex, un bon outil est souvent utile. Il doit permettre de visualiser facilement quelles sont les correspondances, quels sont les groupes constitués, etc. En voici une petite sélection :

- **Regexr** est en AIR mais je lui pardonne... Il permet aussi de sauver ses regex et de consulter une liste notée par les autres internautes (à examiner avec précaution, comme toujours) ;
- **Rubular** est un outil en ligne basé sur les moteurs Ruby (mais c'est très similaire à JavaScript + XRegExp) ;
- **RegexBuddy** est un excellent p'tit outil pas mal foutu qui affiche notamment le détail du fonctionnement des correspondances. En revanche, il a un vrai défaut : il ne marche que sur Windows.

Pour conclure...

Bon, j'ai décidément mis *beaucoup trop de temps* à écrire cet article. Mais il est finalement là, et j'espère qu'il vous aura plu et donné envie d'explorer davantage les possibilités qu'elles offrent pour votre développement au quotidien.

XIV - Envie d'en apprendre davantage ?

Notre atelier **JS Puissant** explore en profondeur tous les aspects techniques avancés du langage JavaScript lui-même. Des méthodes méconnues des types natifs à la programmation fonctionnelle en passant par la *truthiness* et le fonctionnement des prototypes, cette formation JavaScript vous donne un gros coup de boost dans le développement de vos codes JS quels qu'ils soient.

XV - Remerciements

Cet article est publié avec l'aimable autorisation de **Christophe Porteneuve**, l'article original peut être lu sur le blog **JS Attitude : Enfin Maîtriser Les Expressions Rationnelles**.

N'hésitez pas à faire part de vos remarques et commentaires sur le forum ainsi que **sur le blog de l'auteur**.